Système et Réseaux : Partiel 1

mars 2026 — Pierre Rousselin

Consignes et conseils :

- Les prototypes des appels système et de certaines fonctions sont rappelés en fin d'énoncé.
- Pour gagner du temps, sauf indication contraire, vous pourrez supposer que tous les appels système réussissent.
- Les inclusions de fichiers d'en-tête ne sont pas nécessaires non plus.
- Il est inutile de recopier l'énoncé. Lorsque vous devez compléter des programmes, il suffit d'indiquer clairement à quel endroit (numéro de ligne, corps de fonction...) vous ajoutez des instructions ou modifiez le programme.
- La clarté et la simplicité des solutions seront des éléments importants de l'évaluation.
- Le barème est donné à titre indicatif et est susceptible d'être modifié avant la fin du processus.

Exercice 1 : *parallel for* sur 12 points

Une certaine fonction est supposée donnée et sans effets de bord :

```
/* Retourne le résultat d'un long calcul associé à l'argument n */
double compute(int k);
```

1. Écrire un programme `notparallelfor.c` qui :

- met dans une variable `n` de type `int` la valeur entière associée à son unique argument de la ligne de commande, supposé composé de chiffres décimaux
- alloue sur le tas un tableau `results` pouvant stocker `n` valeurs de type `double`
- stocke dans `results[0]` la valeur de `compute(0)`, dans `results[1]` la valeur de `compute(1)`, ..., dans `results[n - 1]` la valeur de `compute(n - 1)`,
- puis, une fois tous les calculs faits, affiche le contenu du tableau, le libère et se termine avec un statut de sortie correspondant à un succès.
- Lorsqu'il n'y a pas un seul argument de la ligne de commande un message d'erreur est affichée sur la sortie appropriée et le processus prend fin avec un statut de sortie approprié.
- Même chose lorsque l'argument de la ligne de commande n'est pas interprétable comme un entier en décimal (utiliser la fonction `sscanf`).

2. Dans un programme nommé `parallelfor.c`, on suppose définie et entière la constante symbolique `NUM_THREADS` (qui vaut par exemple 8). Écrire les modifications à apporter au programme précédent pour faire exécuter les `n` calculs par `NUM_THREADS` *threads* avec les contraintes suivantes :

- le *thread* initial n'appelle pas lui-même la fonction `compute` ; il crée `NUM_THREADS` *threads* et attend qu'ils aient terminé leurs calculs
- le thread numéro `0` calcule `results[0]`, `results[NUM_THREADS]`, `results[2 * NUM_THREADS]`, ...
- le thread numéro `1` calcule `results[1]`, `results[NUM_THREADS + 1]`, `results[2 * NUM_THREADS + 1]`, ...
- Il n'y a pas de variable globale mais il y a un type structuré `struct worker_arg` contenant les champs utiles aux *threads* ; chaque *thread* reçoit l'adresse d'une variable de ce type allouée sur le tas, contenant les données dont elle a besoin pour calculer.
- Lorsque tous les calculs sont terminés, le *thread* initial affiche, comme dans la première version, les éléments du tableau `results`.
- Toute la mémoire allouée sur le tas doit être libérée.

--- * ---

Exercice 2 : `fork`, `wait`, `kill` et `exec` (6 points)

On considère le programme suivant (les inclusions de fichiers d'en-tête ont été omises).

```c
#define N_DFT 3
int main(int argc, char *argv[])
{
        int n = argc > 1 ? atoi(argv[1]) : N_DFT;
        if (n <= 0) {
                printf("der !\n");
                return 0;
        }
        printf("prem's !\n");
        for (; n > 0; n--) {
                pid_t p = getpid();
                printf("lol\n");
                if (fork() != 0) {
                        wait(NULL);
                        printf("plop\n");
                }
                kill(p, SIGKILL);
        }
        execlp("./a.out", "./a.out", "0", NULL);
        printf("haha\n");
        return 0;
}
```

On suppose que le binaire compilé à partir de ce programme s'appelle `a.out` et qu'il est présent dans le répertoire courant. Le signal `SIGKILL` met toujours fin au processus qui le reçoit.

1. (4 points) Représenter l'arbre de création des processus lorsque le programme ci-dessus est exécuté sans argument. Y faire figurer les différents affichages et la mort de chaque processus.
2. (2 points) Donner, en fonction de `n`, le nombre d'affichage de chacune des chaînes de caractères présentes dans le programme lorsque celui-ci est exécuté par la commande `./a.out n` où `n` est un entier strictement positif.

--- * ---

Exercice 3 : Un petit calcul... (10 points)

On considère le programme suivant :

```c
double calcul_a(double x, int n);
double calcul_b(double x, int n);
double calcul_c(double x, int n);
double calcul_d(double x, double y);
double calcul_e(double x);
double calcul_f(double x);

struct calcul {
        double x, a, b, c, d, e;
        int n;
};

int main(void)
{
        struct calcul calc;
        scanf("%lf %d", &(calc.x), &(calc.n));
        calc.a = calcul_a(calc.x, calc.n);
```

```
        calc.b = calcul_b(calc.x, calc.n);
        calc.c = calcul_c(calc.x, calc.n);
        calc.d = calcul_d(calc.a, calc.b);
        calc.e = calcul_e(calc.b + calc.c);
        double res = calcul_f(calc.e - calc.d);
        printf("Résultat : %g\n", res);
        return 0;
}
```

Les fonctions `calcul_a`, ... `calcul_f` sont supposées données et sans effets de bord.

1. (2 points) Donner le diagramme de dépendance entre les différents calculs.

2. (8 points) Sans modifier le type des fonctions `calcul_a`, ..., `calcul_f`, en utilisant des *threads* et éventuellement des sémaphores, changer le programme de façon à paralléliser au maximum les calculs, tout en respectant les dépendances. Le ou les sémaphores pourront être globaux mais pas les autres variables éventuelles. **La simplicité du programme sera prise en compte dans l'évaluation.**

--- * ---

Exercice 4 : Le cuisinier et les mangeurs [1] (10 points)

Un cuisinier a invité `NB_MANGEURS` pour un dîner sans fin chez lui. Ce cuisinier a une marmite qui peut contenir `N` repas et il ne sait cuisiner que `N` repas à la fois.

Quand un mangeur veut manger, il va chercher un repas dans la marmite. Si la marmite est vide, il demande au cuisinier de la remplir.

Quand le cuisinier a fini de cuisiner, il attend qu'on l'appelle pour venir remplir la marmite avec ce qu'il a préparé.

Dans le code qui suit (inclusions de fichiers d'en-tête omises), la marmite est représentée par une pile bornée globale. Lorsqu'un mangeur se sert dans la marmite il dépile un repas et lorsque le cuisinier la remplit (toujours lorsqu'elle est vide), il empile `N` repas. Les fonctions `manger` et `cuisiner` sont données à titre d'exemple mais vous ne ferez aucune hypothèse sur la durée de ces tâches.

```
1  #define N 5
2  int marmite[N];
3  int nb_repas_marmite = 0;
4  void *cuisinier(void *arg);
5  void *mangeur(void *arg);
6  int cuisiner(void);
7  void remplir_marmite(int repas);
8  int se_servir_dans_marmite(void);
9  void manger(int repas);
10 #define NB_MANGEURS 7
11 /* ZONE D'AJOUT 1 */
12 int main(void)
13 {
14         pthread_t mangeurs[NB_MANGEURS], cuisto;
15         int i;
16         /* ZONE D'AJOUT 2 */
17         pthread_create(&cuisto, NULL, cuisinier, NULL);
18         for (i = 0; i < NB_MANGEURS; i++)
19                 pthread_create(&mangeurs[i], NULL, mangeur, NULL);
20         /* ZONE D'AJOUT 3 */
21         return 0;
22 }
23 int cuisiner(void) { return rand() % 10; }
24 void remplir_marmite(int repas)
```

1. d'après un problème de Greg Andrews

```
25 {
26         int i;
27         for (i = 0; i < N; i++)
28                 marmite[nb_repas_marmite++] = repas;
29 }
30 int se_servir_dans_marmite(void) { return marmite[--nb_repas_marmite]; }
31 void manger(int repas) { printf("Miam, quel bon %d !\n", repas); }
32 void *cuisinier(void *arg)
33 { /* À MODIFIER 1 */
34         for (;;) {
35                 int repas = cuisiner();
36                 remplir_marmite(repas);
37         }
38 }
39 void *mangeur(void *arg)
40 { /* À MODIFIER 2 */
41         for (;;) {
42                 int repas = se_servir_dans_marmite();
43                 manger(repas);
44         }
45 }
```

1. Ajouter les instructions permettant au *thread* initial d'attendre la fin de tous les *threads* créés (on fera « comme si » ces *threads* se terminent à un moment).
2. En ajoutant sémaphore(s) et/ou mutex et/ou variable(s) globale(s), synchroniser cuisinier et mangeurs comme indiqué ci-dessus. Les mutex et sémaphores créés devront être détruits (en faisant encore une fois « comme si » cuisinier et mangeurs se terminent à un moment).

   Préserver au maximum le parallélisme (par exemple le cuisinier doit pouvoir cuisiner pendant que les mangeurs se servent, les mangeurs doivent pouvoir manger en même temps, ...).

   Indication : seul un mangeur à la fois peut avoir accès à la marmite.

--- * ---

Prototypes des fonctions et appels systèmes à utiliser

Certains arguments sont mis à `NULL` ou `0` car nous n'avons pas vu leurs rôles en cours.

```
/* Fonctionne comme scanf mais lit la chaîne str au lieu de l'entrée
   standard ; retourne le nombre d'entrées correctement associées dans str */
int sscanf(const char *restrict str, const char *restrict format, ...);
void *malloc(size_t n);
void free(void *p);
pid_t fork(void);
pid_t wait(int *wstatus);
int execlp(const char *file, const char *arg, ..., NULL);
int kill(pid_t pid, int sig);
int pthread_create(pthread_t *thread, NULL,
                       void *(*start_routine) (void *), void *arg);
int pthread_join(pthread_t thread, void **retval);
int pthread_mutex_init(pthread_mutex_t *mutex, NULL);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int sem_init(sem_t *sem, 0, unsigned int valeur);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);
```